

StatesAndRates

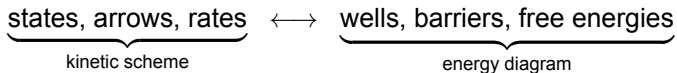
Graphs of states connected by rates, and the linear algebra that solves them

A small package for building a kinetic mechanism once and asking it every question: equilibrium, relaxation, coarse graining, passage times, dwell times, single molecules.

<https://github.com/lanmelo/StatesAndRates>

A mechanism is a hypothesis

Not a summary of data — a **claim**: which states exist, which interconvert, how fast.



one claim, drawn twice — wherever detailed balance holds

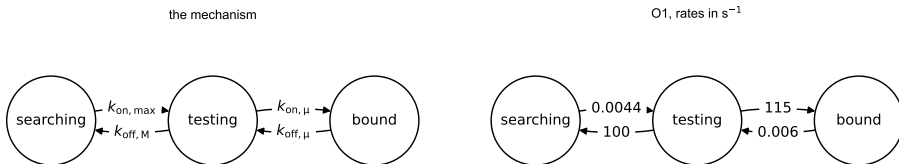
So it predicts things an experiment can contradict:

- is a dissociation curve one exponential or two, and with what amplitudes?
- does a substitution move a well or a barrier — association, or retention?
- what does an assay that cannot resolve the intermediate report as “the” off-rate?
- how often is an escape recaptured before it counts as one?

Each is one operation on one matrix.

The one object

A **Graph** is a set of states and the first-order rate constants between them. Everything else is a function of it.



Rate constants are stored as **logarithms**, for three reasons:

- they span many orders of magnitude (10^{-3} to 10^2 s^{-1} here);
- a free-energy perturbation is then an *additive* shift;
- an absent transition is $-\infty$, so structure and values live in one array.

A **Scheme** is the same topology with the rates *named* rather than valued — the mechanism before any numbers.

From arrows to a matrix

A rate constant $k_{i \rightarrow j}$ is a **probability per unit time**: in a short interval dt , a molecule in state i moves to j with probability $k_{i \rightarrow j} dt$, regardless of how long it has already been in i . That single memoryless assumption fixes everything else:

- the time spent in i before *any* jump is exponentially distributed with rate $\sum_j k_{i \rightarrow j}$ — the *total* rate out;
- the destination is then chosen with probability $k_{i \rightarrow j} / \sum_{j'} k_{i \rightarrow j'}$, independently of the waiting time.

That is all “continuous-time Markov chain” means here. Nothing below needs more probability theory than this — the rest is linear algebra on the matrix the diagram already is:

one arrow $i \rightarrow j \iff$ one off-diagonal entry $k_{i \rightarrow j}$,

with the diagonal fixed by bookkeeping, as the generator slide sets out.

What first order buys, and what it costs

Buys: every rate is a constant, so the master equation on the next slide is *linear*. That is what makes every question in this talk one operation on one matrix, with no time stepping and no iteration.

Costs: association is bimolecular — a protein and a site have to find each other — so it must enter as a **pseudo-first-order** rate with the concentration folded in:

$$k_{\text{on,max}} [\text{s}^{-1}] = k_{\text{on}}^{(2)} [\text{M}^{-1}\text{s}^{-1}] \times [\text{X}].$$

- So every result belongs to *that* concentration, not to a standard state. Free energies come out as ΔG , not ΔG° .
- k_{d} is unaffected by concentration; k_{a} scales with it. That asymmetry is why measured off-rates transfer between experiments and on-rates do not.
- Valid while the free pool is *buffered*, $[\text{X}] \gg [\text{sites}]$. Depletion, competition and titration need a genuinely bimolecular treatment.

Notation, once

- $p(t)$ — the **state vector**. Entry $p_i(t)$ is how much is in state i at time t : a probability for one molecule, or a population for many. Either way the total is conserved, and it is what every time course in this talk plots.
- π — the **equilibrium distribution**: the p that has stopped changing. Entry π_i is the equilibrium population of state i . Bold is the whole vector and a subscript picks out one entry, here and throughout: p and p_i , K and K_{ij} .
- K — the **rate matrix** read straight off the diagram: $K_{ij} = k_{i \rightarrow j}$ off the diagonal, $K_{ii} = 0$.
- $\mathbf{1}$ — the **all-ones vector**. So $K\mathbf{1}$ is the vector of *row sums*, whose i th entry $\sum_j k_{i \rightarrow j}$ is the total rate out of state i — the quantity that already appeared as the exit rate on the last slide.
- $\text{diag}(v)$ — the matrix with v down its diagonal and zeros elsewhere.

Multiplying by $\mathbf{1}$ is the recurring trick: it sums along rows. “ $Q\mathbf{1} = \mathbf{0}$ ” will mean “every row of Q sums to zero”, and the $-\mathbf{1}$ in the passage-time solve later is the constant on the right of one equation per state.

The generator

Take the rate matrix and put minus each row's own sum on its diagonal:

$$Q = K - \text{diag}(K\mathbf{1}), \quad Q_{ij} = k_{i \rightarrow j} \ (i \neq j), \quad Q_{ii} = - \sum_{j \neq i} k_{i \rightarrow j}.$$

For the searching/testing/bound chain of slide 2, that is

$$Q = \begin{pmatrix} -k_{\text{on,max}} & k_{\text{on,max}} & 0 \\ k_{\text{off,M}} & -(k_{\text{off,M}} + k_{\text{on},\mu}) & k_{\text{on},\mu} \\ 0 & k_{\text{off},\mu} & -k_{\text{off},\mu} \end{pmatrix}.$$

- Off-diagonal entries are the arrows. Zeros are the arrows that do not exist — here searching and bound do not interconvert directly.
- Each diagonal entry is minus the rest of its row, so $Q\mathbf{1} = 0$: **every row sums to zero.**

The master equation

Count what flows into a state and what flows out of it:

$$\frac{dp_i}{dt} = \underbrace{\sum_{j \neq i} k_{j \rightarrow i} p_j}_{\text{arriving from every other state}} - p_i \underbrace{\sum_{j \neq i} k_{i \rightarrow j}}_{\text{leaving } i \text{ by every route out}}$$

That is the i th column of Q acting on p , so the whole system is one matrix equation:

$$\frac{dp}{dt} = Q^T p.$$

- The transpose is there because Q_{ij} is the rate *out of* i , while dp_i/dt needs the rates *into* it.
- Zero row sums make Q **singular** — that is conservation, $\mathbf{1}^T dp/dt = 0$.
- Everything from here is one linear-algebra question about Q : a null vector, a matrix exponential, an eigenproblem, or a linear solve — which one depends on what the experiment measures.

Detailed balance: when do free energies exist?

Detailed balance is the statement that every transition is individually balanced at equilibrium,

$$\pi_i k_{i \rightarrow j} = \pi_j k_{j \rightarrow i} \iff \ln \frac{\pi_j}{\pi_i} = \ln \frac{k_{i \rightarrow j}}{k_{j \rightarrow i}}.$$

Summing that around any cycle must give zero — **Kolmogorov's criterion**. A graph that violates it is being driven and has no free-energy landscape.

How it is tested. Not by enumerating cycles. Walk a spanning tree, accumulating $\ln \pi$ from the ratios; that fixes $\ln \pi$ up to a constant using $n - 1$ of the transitions. Then the residual

$$r_{ij} = \ln \pi_j - \ln \pi_i - \ln \frac{k_{i \rightarrow j}}{k_{j \rightarrow i}}$$

must vanish on *every remaining* transition. One tree walk, $O(n^2)$, no cycle basis.

Equilibrium: a null vector, computed carefully

The equilibrium distribution is the stationary solution,

$$Q^T \pi = 0, \quad \sum_i \pi_i = 1.$$

Q is singular by construction, so forming this as a least-squares or nullspace problem invites catastrophic cancellation: the diagonal is a sum of the off-diagonals, and subtracting nearly equal large numbers destroys the small ones.

GTH reduction instead eliminates one state at a time using *only the non-negative off-diagonal rates*:

$$Q_{ik} \leftarrow \frac{Q_{ik}}{\sum_{j < k} Q_{kj}}, \quad Q_{ij} \leftarrow Q_{ij} + Q_{ik} Q_{kj} \quad (i, j < k),$$

then back-substitutes $w_k = \sum_{i < k} w_i Q_{ik}$ from $w_0 = 1$ and normalises. No subtraction ever occurs, so the result is accurate even when populations differ by many orders of magnitude.

Free energies and energy diagrams

From the equilibrium distribution,

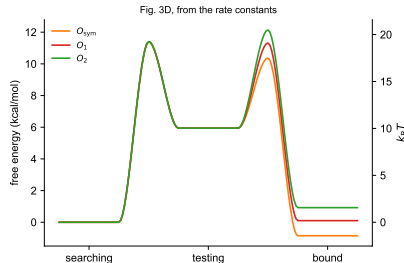
$$G_i = -RT \ln \pi_i,$$

and from transition-state theory, writing

$$k_{i \rightarrow j} = A e^{-(G_{ij}^\ddagger - G_i)/RT},$$

$$G_{ij}^\ddagger = G_i - RT \ln \frac{k_{i \rightarrow j}}{A}.$$

- Both directions must give the *same* barrier, which is detailed balance — so the package refuses without it. The prefactor A shifts every barrier equally, leaving all *differences* unchanged.



Same $k_{off, \mu}$, so the same well depth from the barrier; different recognition, different wells.

These are ΔG , *not* ΔG° : association enters as $k_{on}[TF]$, so this is drawn at that concentration. The standard-state shift is 12 kcal/mol.

Perturbations are additive in log space

A Mutation is expressed in kcal/mol on the *landscape*, not on individual rates, and applied as $\ln k \leftarrow \ln k + \Delta$:

$$\underbrace{\Delta_{ij} += \frac{\Delta G_i}{RT} \quad \forall j}_{\text{raise a well}} \quad \underbrace{\Delta_{ij}, \Delta_{ji} -= \frac{\Delta G_{ij}^\ddagger}{RT}}_{\text{raise a barrier}}$$

- Raising a well speeds up *every* transition out of it — one row of the matrix.
- Raising a barrier slows *both* directions equally, so the equilibrium constant is untouched. Either way the graph stays thermodynamically consistent.
- $-\infty + \text{finite} = -\infty$: perturbing an absent transition does nothing, for free.
- Mutations add with +, so a double mutant is the sum of two single ones — and that is testable against applying them in turn.

Solving it: the matrix exponential

$dp/dt = Q^T p$ is the matrix version of $\dot{x} = ax$, and it has the same solution:

$$p(t) = e^{Q^T t} p(0), \quad e^A \equiv I + A + \frac{1}{2}A^2 + \dots$$

So no time stepping is needed — the answer at $t = 10^6$ costs the same as at $t = 1$.

But the series is not how you compute it. Software squares a scaled exponential repeatedly, and each squaring doubles any error already present. That becomes visible once the largest rate times the longest time exceeds roughly 10^9 .

Master equations hit that constantly, because they are **stiff** whenever the timescales are well separated — and well-separated timescales are usually the point. In the model here the fast mode is 10^4 times the slow one, so following the slow relaxation means carrying the fast one for 10^4 of its own lifetimes.

When the graph is reversible there is a way out that costs nothing.

Solving it exactly: symmetrise first

Q is not symmetric, so its eigenvectors are not orthogonal and diagonalising it is ill-conditioned. Detailed balance repairs that. Conjugate by the square root of the equilibrium distribution, $\Pi = \text{diag}(\pi)$:

$$S = \Pi^{1/2} Q \Pi^{-1/2}, \quad S_{ij} = \sqrt{\frac{\pi_i}{\pi_j}} k_{i \rightarrow j}.$$

Now use $\pi_i k_{i \rightarrow j} = \pi_j k_{j \rightarrow i}$, which gives $\pi_i / \pi_j = k_{j \rightarrow i} / k_{i \rightarrow j}$:

$$S_{ij} = \sqrt{\frac{k_{j \rightarrow i}}{k_{i \rightarrow j}}} k_{i \rightarrow j} = \sqrt{k_{i \rightarrow j} k_{j \rightarrow i}} = S_{ji}.$$

- π **cancels**. The symmetrised matrix is just the geometric mean of each forward and reverse rate — no equilibrium calculation needed to build it.
- It is symmetric, so `eigh` gives a real spectrum and genuinely orthogonal modes, at any stiffness.

Solving it exactly: putting it back

Conjugation does not change eigenvalues, so S and Q have the same spectrum. Diagonalise the easy one, $S = V \Lambda V^\top$, and undo the conjugation:

$$p(t) = \Pi^{1/2} V e^{\Lambda t} V^\top \Pi^{-1/2} p(0).$$

Read right to left, that is four steps: weight the start by $\Pi^{-1/2}$, project onto the modes, decay each mode by its own $e^{\lambda t}$, then transform back.

- $e^{\Lambda t}$ is diagonal — one scalar exponential per mode — so nothing is squared and nothing amplifies. It is exact however stiff the rates and however long the time.
- The cost is one eigendecomposition, reused for every requested time.
- Only the eigenvalues are needed if all you want is the *rates* of the phases, which is the next slide.

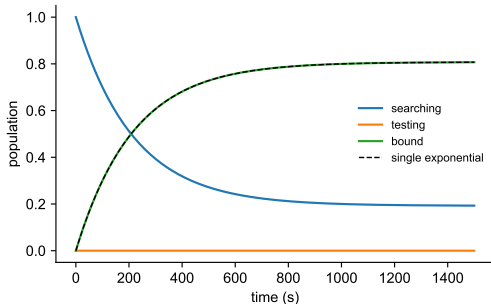
The relaxation spectrum

The eigenvalues of $-S$ are the relaxation rates:

$$0 = \lambda_0 < \lambda_1 \leq \dots \leq \lambda_{n-1},$$

λ_0 belonging to equilibrium and the rest being the rates of the exponential phases of *any* relaxation. Their reciprocals are the relaxation times. For this chain, $\lambda_{\pm} = \frac{1}{2}(T \pm \sqrt{T^2 - 4D})$, with $T = \sum_i k_i$ and $D = k_{\text{off},M}k_{\text{off},\mu} + k_{\text{off},\mu}k_{\text{on},\text{max}} + k_{\text{on},\text{max}}k_{\text{on},\mu}$.

- A three-state chain has two non-zero modes, but here they are 10^4 apart, so the slow one is all an experiment sees.
- That is why the population curve is indistinguishable from a single exponential with rate $k_a + k_d$ — and it is the justification for coarse graining.

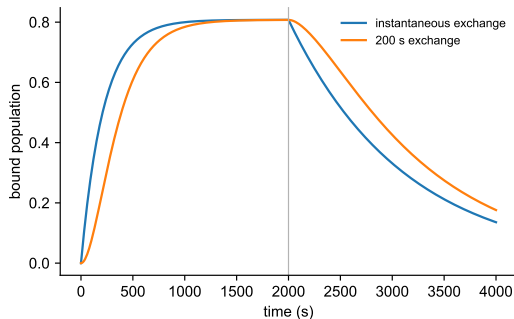


Populations from propagate; dashed is one exponential at $1/(k_a + k_d)$.

Absorbing states: the dissociation experiment

Setting one column of the rates to $-\infty$ in log space makes a state **absorbing**. Nothing rebinds, so the population only drains out of it.

- That is a wash step: protein removed from solution.
- The graph is no longer reversible, so propagate falls back to the matrix exponential — correctly, since detailed balance no longer holds.
- Survival curves, and therefore measured off-rates, are exactly this calculation.



Association, then the searching state made absorbing at $t = 2000$ s.

Coarse graining: two different reductions

An experiment rarely resolves every state. Lumping states $\mathcal{A}, \mathcal{B}, \dots$ gives a graph over macrostates — but there is more than one right answer, because the answer depends on what the instrument measures.

Local-equilibrium average (`coarse_grain`):

$$K_{\mathcal{A}\mathcal{B}} = \sum_{i \in \mathcal{A}} w_i \sum_{j \in \mathcal{B}} k_{i \rightarrow j}, \quad w_i = \frac{\pi_i}{\sum_{i' \in \mathcal{A}} \pi_{i'}}.$$

Reproduces the equilibrium populations and the equilibrium *fluxes* exactly.

Passage-time inverse (`passage_rates`):

$$K_{\mathcal{A}\mathcal{B}} = \left(\sum_{i \in \mathcal{A}} w_i m_i(\mathcal{B}) \right)^{-1},$$

the reciprocal of a mean first passage time. This is the rate a measured *dwelt time* reports.

They agree only when the partition is **lumpable**: every state of $\mathcal{A}$ has the same total rate into $\mathcal{B}$. Then the macro kinetics is genuinely Markovian.

Mean first passage time: the first-step argument

How long, on average, until the system first reaches a target set $\mathcal{T}$? Let m_i be that time starting from state i , with $m_i = 0$ for $i \in \mathcal{T}$.

Take one step and average. From i you wait, then land somewhere:

$$m_i = \underbrace{\frac{1}{\sum_j k_{i \rightarrow j}}}_{\text{mean wait in } i} + \sum_j \underbrace{\frac{k_{i \rightarrow j}}{\sum_{j'} k_{i \rightarrow j'}}}_{\text{chance of landing in } j} \underbrace{m_j}_{\text{time still to go}}.$$

Multiply through by the total exit rate and move the m terms to one side. The waiting time and the branching probabilities recombine into the generator, and every trace of probability disappears:

$$\boxed{\sum_{j \notin \mathcal{T}} Q_{ij} m_j = -1} \quad (i \notin \mathcal{T}).$$

One linear equation per non-target state, with the target states simply absent — they contribute nothing because $m_j = 0$ there.

Mean first passage time: the linear solve

In matrix form, restricting to the non-target (“free”) states $\mathcal{F}$:

$$Q_{\mathcal{F}\mathcal{F}} m_{\mathcal{F}} = -\mathbf{1} \quad \Rightarrow \quad m_{\mathcal{F}} = -Q_{\mathcal{F}\mathcal{F}}^{-1} \mathbf{1}.$$

With $\mathcal{F} = \{\text{testing, bound}\}$, this is slide 7’s matrix with the searching row and column struck out:

$$Q_{\mathcal{F}\mathcal{F}} = \begin{pmatrix} -(k_{\text{off},M} + k_{\text{on},\mu}) & k_{\text{on},\mu} \\ k_{\text{off},\mu} & -k_{\text{off},\mu} \end{pmatrix}, \quad m_{\mathcal{F}} = \begin{pmatrix} k_{\text{off},\mu} + k_{\text{on},\mu} \\ k_{\text{off},M} + k_{\text{off},\mu} + k_{\text{on},\mu} \end{pmatrix} / k_{\text{off},M} k_{\text{off},\mu}.$$

- The full Q is singular, so this could not be solved as written on all states. **Deleting the target rows and columns is exactly what makes it invertible** — the deleted rows are where the probability leaks out.
- $-Q_{\mathcal{F}\mathcal{F}}^{-1}$ is the **fundamental matrix**: its (i, j) entry is the expected *total time spent in j* before absorption, starting from i . So m is just its row sums, and $-Q_{\mathcal{F}\mathcal{F}}^{-1} \mathbf{1}$ says “add up the time spent everywhere”.
- One solve, $O(n^3)$, gives the passage time from *every* state at once — not one target at a time.
- Everything downstream is this one inverse: the macroscopic rates of the next slide, and the observable dwell times later.

The same inverse, read two ways

The first-step argument gets to $m = -Q_{\mathcal{FF}}^{-1}\mathbf{1}$ through probabilities. There is a route with no probability in it at all.

The chance of still being free at time t , having started in i , is $\Phi_i(t) = \sum_{j \in \mathcal{F}} (e^{Q_{\mathcal{FF}}^T t})_{ji}$, and a mean lifetime is the integral of a survival curve:

$$m_i = \int_0^\infty \Phi_i(t) dt, \quad \int_0^\infty e^{Q_{\mathcal{FF}}^T t} dt = -Q_{\mathcal{FF}}^{-1},$$

the matrix analogue of $\int_0^\infty e^{-at} dt = 1/a$. It converges precisely because absorption makes every eigenvalue negative.

So the fundamental matrix is the **time-integrated propagator**: its (i, j) entry is the total time spent in j , the *mean residence time*, and the passage time is the sum over j . That is the derivation in Bar-Haim & Klafter, *J. Chem. Phys.* **109**, 5187 (1998).

Both are available: `mean_first_passage_times` for the totals in one solve, `mean_residence_times` for the matrix. Divide an entry by the mean time per visit and it becomes a *visit count* — $1/(1 - p_{\text{tot}})$ bound visits per escape, exactly.

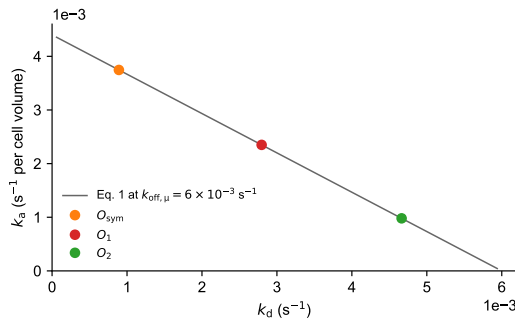
Recovering the macroscopic rates

For Marklund's model, lumping the invisible testing state and taking passage times between searching and bound reproduces the paper's Eq. 1 exactly:

$$k_a = k_{\text{on},\text{max}} p_{\text{tot}}, \quad k_d = k_{\text{off},\mu} (1 - p_{\text{tot}}),$$

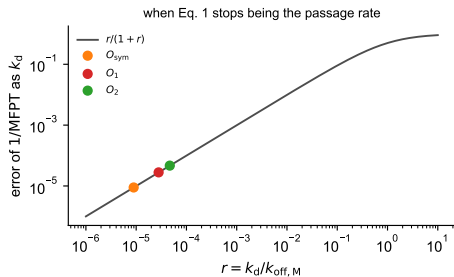
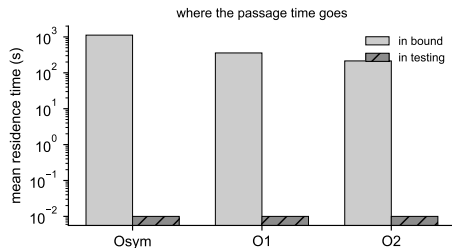
with $p_{\text{tot}} = k_{\text{on},\mu} / (k_{\text{on},\mu} + k_{\text{off},\text{M}})$ the recapture probability.

- No fitting, and no symbolic algebra either — the package has none. This is the linear solve, which *agrees* with Eq. 1 rather than deriving it.
- The three operators differ only in p_{tot} , and they land on the predicted curve.



Line: Eq. 1 swept over p_{tot} . Points: the three operators, from passage times.

Splitting the passage time recovers Eq. 1 — and its limits



The residence matrix splits it: $m_b = \underbrace{R_{bb}}_{1/k_d} + \underbrace{R_{bt}}_{1/k_{off,M}}$. There are $1/(1 - p_{tot})$ visits to bound,

each lasting $1/k_{off,\mu}$, so their product is Marklund's $1/k_d$ **exactly**. Testing is visited as often, but each visit lasts $1/(k_{on,\mu} + k_{off,M})$ and the factors cancel, leaving one intermediate lifetime. So $1/\text{MFPT}$ is Eq. 1 only when that lifetime is short: the error is exactly $r/(1 + r)$ with $r = k_d/k_{off,M}$.

Why simulate at all?

Everything so far computes what the population does **on average**. $p(t)$ is a distribution over states, not a molecule.

That is enough for a bulk time course, an equilibrium constant, or a relaxation rate. It is not enough when the observable is something the mean does not determine:

- the *distribution* of dwell times, not just its mean — is it one exponential or two?
- how many times a molecule revisits a state before it escapes;
- what a single-molecule trace looks like, which is what a microscope records;
- any statistic of a trajectory: waiting-time correlations, first passages beyond the mean, rare excursions.

A trajectory is one realisation of the *same* generator. Nothing new is assumed — and averaging many of them must reproduce `propagate`, which is a useful check on both.

One step of the simulation

Every transition is first order, so molecules never interact: each is an independent copy of the same process and they advance together. Simulating 200 means 200 *independent* copies, not 200 molecules competing for one site.

In state i , ask each outgoing arrow when it would fire:

$$\tau_{i \rightarrow j} = \frac{E_j}{k_{i \rightarrow j}}, \quad E_j \sim \text{Exp}(1) \text{ drawn independently,}$$

an exponential waiting time of rate $k_{i \rightarrow j}$, which is what that rate constant means. Then take the **first** arrow to fire:

$$j^* = \arg \min_j \tau_{i \rightarrow j}, \quad t \leftarrow t + \min_j \tau_{i \rightarrow j}, \quad i \leftarrow j^*.$$

Repeat — hence *first-reaction* method.

The method is not the restriction. Gillespie's algorithm is exact for any well-mixed mass-action network, bimolecular included: for $A + B \rightarrow C$ the propensity is $c N_A N_B$, recomputed from the current counts each step. It is *this package* that is first order, which is what lets it track independent molecules on a fixed graph rather than a vector of counts.

Why that is the right answer

Two facts make the shortcut legitimate:

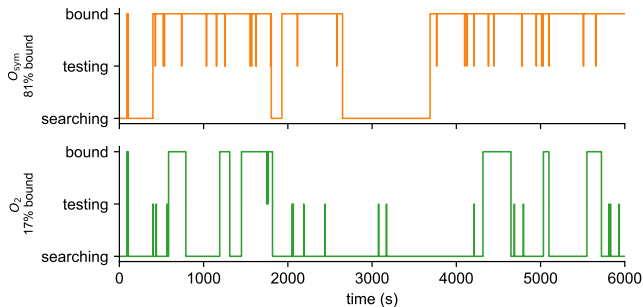
- the minimum of independent exponentials is exponential with the *summed* rate, $\min_j \tau_{i \rightarrow j} \sim \text{Exp}(\sum_j k_{i \rightarrow j})$ — which is the exit time the memoryless assumption demanded;
- the probability that arrow j is the one that fires first is $k_{i \rightarrow j} / \sum_{j'} k_{i \rightarrow j'}$ — which is the branching probability.

So drawing a race between the arrows reproduces “sample the exit time, then choose a destination” without ever writing those two steps separately.

Done in log space. $\ln \tau_{i \rightarrow j} = \ln E_j - \ln k_{i \rightarrow j}$, and the rates are already stored as logarithms. Rates spanning many orders of magnitude therefore cost no accuracy, and an absent transition is $\ln k = -\infty$, giving $\tau = +\infty$ rather than a division by zero.

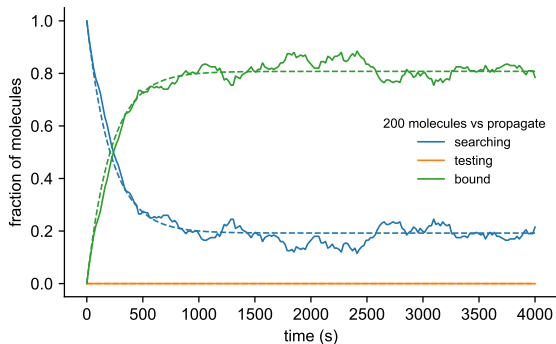
An **absorbing** state has no finite τ at all, so the molecule simply stops — no special case needed.

What a single molecule looks like



- Same mechanism, two operators. O_{sym} sits bound; O_2 keeps returning to the search.
- The trajectory carries every jump, so occupancy at arbitrary times is a `searchsorted` plus a cumulative sum — no re-simulation.
- The check on all the matrix algebra: the ensemble average of many trajectories reproduces `propagate`.

The ensemble average is the master equation



Solid: 200 molecules through occupancy;
dashed: propagate on the same graph.

- This is the one place the two halves of the package have to agree, and they share no code: NumPy sampling of waiting times against a JAX eigendecomposition.
- The residual is sampling noise, not bias: rms 0.026 against the binomial $1/\sqrt{N} = 0.071$, and it shrinks with more molecules, not with a finer grid.
- One molecule at a time and a whole population measure the same object; which to run is a question about noise.

Dwell times: two rates, and they are not the same

A dwell in one state can report either of two rates.

Per visit — arrival until the next jump out, whatever the destination:

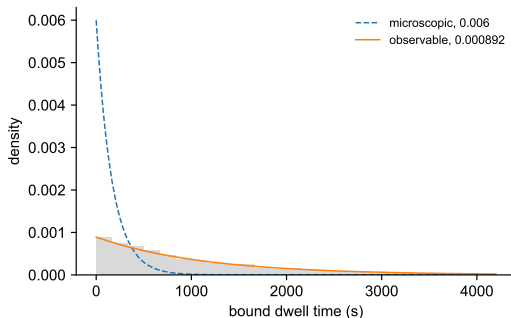
exponential with $k_{\text{micro}} = \sum_j k_{i \rightarrow j}.$

Until it actually leaves — arrival until first reaching a target, so excursions that come straight back *do not* end the dwell:

$$k_{\text{obs}} = 1/m_i(\mathcal{T}).$$

- The second is always slower, by one over the probability that an escape is *not* recaptured.
- Which one an experiment measures depends on whether it can see the excursion. If the intermediate is invisible, it measures the passage time — so the microscopic rate is not what a dwell histogram gives.
- This is the microscopic/macroscopic distinction, in one state.

Dwells, sampled and predicted



- Histogram: Osym dwells extracted from a Gillespie run. Lines: $k e^{-kt}$ at the two rates above, computed from the graph, not fitted to the sample.
- The sample follows the *observable* rate, not the microscopic one — $1/k_{\text{off},\mu} = 167$ s against a measured mean of ~ 1100 s for Osym.
- Dwells still running at the end are dropped, pulling the sample mean slightly low.

Time-dependent rates, and derivatives

When the rates move. A ligand jump, a flow ramp, a wash with finite mixing: then $Q = Q(t)$ and there is no closed form. The package integrates

$$\frac{d\mathbf{p}}{dt} = \mathbf{Q}(t)^\top \mathbf{p}$$

with an implicit solver (Kvaerno5), because master equations are stiff whenever their timescales are well separated — which is whenever the problem is interesting.

Why it is all differentiable. A graph is registered as a JAX pytree whose *only* leaf is the log-rate matrix; the state names and temperature are static metadata. So

$$\frac{\partial}{\partial \ln k_{i \rightarrow j}} (\text{any observable})$$

comes from `jax.grad`, and a library of variants is one `jax.vmap`. Fitting a mechanism to data is then the same machinery as evaluating it.

Everything above, as matrix algebra

One operator, two directions. **Forward** questions act on a distribution and need the rates *into* a state, so they carry $Q^\top$; **backward** questions act on a function of where you *started* and need the rates *out*, so they carry Q , with the target's rows and columns deleted.

	Unknown	Equation	Name	What is measured
Forward	π	$Q^\top \pi = 0, \mathbf{1}^\top \pi = 1$	stationary distribution	occupancy, K_d , ΔG
	$p(t)$	$p(t) = e^{Q^\top t} p(0)$	propagation	a kinetic trace
	J	$J_{ij} = \pi_i k_{i \rightarrow j} - \pi_j k_{j \rightarrow i}$	stationary flux	turnover, dissipation
Backward	N	$Q_{\mathcal{F}\mathcal{F}} N = -I$	fundamental matrix	where the time goes
	m	$Q_{\mathcal{F}\mathcal{F}} m = -\mathbf{1}, m = N\mathbf{1}$	MFPT	dwelt time, $1/k_{\text{off}}$
	H	$Q_{\mathcal{F}\mathcal{F}} H = -Q_{\mathcal{F}\mathcal{A}}$	splitting probabilities	branching ratio
Either	λ	$Qv = \lambda v$	relaxation spectrum	exponential phases

The two halves are one object read twice: $-Q_{\mathcal{F}\mathcal{F}}^{-1} = \int_0^\infty e^{Q_{\mathcal{F}\mathcal{F}} t} dt$, and $e^{Q^\top t} \rightarrow \pi \mathbf{1}^\top$ as $t \rightarrow \infty$. So a backward solve is a forward propagation with the clock integrated out — no t in it, and every starting state at once — and equilibrium is propagation at $t \rightarrow \infty$, only cheaper.

The map

Question	Linear algebra	Entry point
Is there a landscape?	tree walk + residual	<code>is_detailed_balanced</code>
Equilibrium populations	null vector of Q^T , by GTH	<code>equilibrium</code>
Free energies, barriers	$-RT \ln \pi$; $G_i - RT \ln(k/A)$	<code>free_energies</code> , <code>barrier_energies</code>
Perturb the landscape	additive shift of $\ln k$	<code>Mutation</code> , <code>Graph.mutate</code>
Populations over time	$e^{Q^T t}$, or $\Pi^{1/2}$ -conjugated eigh	<code>propagate</code>
Observable phases	eigenvalues of $-S$	<code>relaxation_rates</code> , <code>...times</code>
Driven rates	implicit ODE solve	<code>integrate</code>
What a lumped experiment sees	flux average, or $1/\text{MFPT}$	<code>coarse_grain</code> , <code>passage_rates</code>
Time to reach a state	$-Q_{\mathcal{F}\mathcal{F}}^{-1} \mathbf{1}$	<code>mean_first_passage_times</code>
Where that time is spent	$-Q_{\mathcal{F}\mathcal{F}}^{-1}$ itself	<code>mean_residence_times</code>
Is lumping legitimate?	equal row sums into each block	<code>is_lumpable</code>
Individual molecules	first-reaction sampling	<code>simulate</code>
Dwell distributions	row sum vs. MFPT	<code>Trajectory.dwells</code> , <code>dwell_rates</code>
Draw it	—	<code>state_diagram</code> , <code>energy_diagram</code>

Build the mechanism once as a `Scheme`; fill it with numbers to get a `Graph`; ask any of the above.